TEMA 1

- Algoritmo: conjunto de pasos para resolver un problema, que no tiene que ser necesariamente informático. Es la idea, como una receta de cocina: dice qué hay que hacer, pero no en qué lenguaje se escribe. Se divide en entrada → proceso → salida.

- Pseudocódigo: representa, mediante una serie de pasos, la solución al algoritmo con palabras parecidas a las usadas en programación, pero que cualquier persona entiende (con lenguaje natural). Ejemplo de pseudocódigo:

1. Pedir num1
2. Pedir num2
3. suma ← num1 + num2
4. mostrar suma

- Depurar: ir viendo paso a paso, línea a línea, como se comporta el código.

- Programa informático: secuencia de instrucciones lógicas para llegar a la resolución de un problema, o en otras palabras, es la **implementación** de uno o más algoritmos, la receta anterior pero detallada.

- Software: es un conjunto de programas informáticos que trabajan juntos para un fin: una calculadora completa, un sistema operativo, un paquete de ofimática, un videojuego…

Para que un programa se pueda ejecutar, necesita del **hardware** y del **software** (sistema operativo) de nuestro ordenador. El **SO** debe cargar el programa (sus datos e instrucciones) en la **memoria principal (RAM)** y de ahí pasa a la **CPU**.

LOS REQUISITOS DE HARDWARE

Todo ordenador sigue la **arquitectura Von Neumann**: memoria principal, sistema de entrada/salida y CPU, formada por la **ALU** (unidad aritmético-lógica), y los **registros** y la **unidad de control**.

Las capas: el **usuario** usa las **aplicaciones**, estas se apoyan en el **SO** y este gobierna el **hardware**.

¿Por qué hay que traducir?

La CPU solo ejecuta lenguaje máquina. Todos los programas se desarrollan en un lenguaje de programación de **alto nivel**. La CPU es quien ejecuta los programas, pero utiliza lenguaje de **bajo nivel** (lenguaje máquina). Por tanto, será necesaria una **traducción** del lenguaje de alto nivel a lenguaje máquina antes de ejecutar. El camino completo: el programador escribe el **código fuente**; el **compilador o intérprete** lo traduce (son traductores); el **cargador** lo lleva al procesador, que lo ejecuta.

Hay tres tipos de niveles: lenguaje de **bajo nivel o máquina**, lenguaje de **medio nivel o ensamblador** y lenguaje de **alto nivel**.

El lenguaje ensamblador puede considerarse como un paso hacia el programador, pero sin soltar la máquina. Se utilizó como sustituto del lenguaje máquina. Se utilizan instrucciones más sencillas, pero no son como el lenguaje natural. **Necesita compilarse y traducirse** a lenguaje máquina. Se trabaja con **registros del procesador** y **direcciones de memoria**. Ejemplo: lenguaje de máquina del Intel 8088.

Dónde se programa hoy: lenguaje de alto nivel, cercana al usuario. En esta categoría está la mayoría de los lenguajes actuales. Se aprecian palabras reservadas como **while**, **do**, **if**, **for**... dependiendo del lenguaje. Ofrecen **librerías y funciones** para solucionar problemas ya resueltos. También utilizan ***frameworks***: esquema o marco de trabajo que ofrece una estructura base para elaborar un proyecto. Ejemplos: Java, C/C++, Python...

Valor absoluto Math.abs en Java: los números positivos se quedan como tal, y los negativos, se convierten en positivos “-(-2) = 2”.

PARADIGMA DE PROGRAMACIÓN

Un paradigma es la forma en que se utiliza un lenguaje de programación: cómo se plantea la solución a un problema.

No depende solo del lenguaje: el mismo lenguaje puede usarse de más de una manera. Existen dos grandes paradigmas: **imperativo** y **declarativo**.

Imperativo: el **cómo** la máquina debe hacer el trabajo, **paso a paso**. La mayoría de lenguajes usan este paradigma. Dentro de este podemos encontrar la **Programación Orientada a Objetos**, la programación por procedimientos. *En nuestro pseudoprograma de suma: LEER num1, suma = num1 + num2... son instrucciones concretas; eso es programar de forma imperativa.*

Declarativo: el qué. No se describen los pasos para resolver el problema, sino **el resultado o las condiciones que quieres obtener**. El sistema se encarga de determinar cómo conseguirlo, no le damos las instrucciones para proceder, solo le decimos lo que queremos conseguir. Dentro de este podemos encontrar el lenguaje **SQL**, la programación funcional, la programación lógica, etc. Ejemplo en SQL:

```
SELECT titulo

FROM juegos

WHERE valoracion > 9;
```

En esta consulta se dice **qué filas** queremos, el motor interno decide los pasos para conseguirlas.

Java es declarativo e imperativo al mismo tiempo, lo que lo convierte en **multiparadigma**.

MULTIPARADIGMA

Un lenguaje **multiparadigma** ofrece más de una forma (paradigma) de trabajar. Python, al igual que Java, es otro ejemplo de ello. Tanto el uno como el otro son Programación Orientada a Objetos.

TRADUCTORES

Traductor: programa que se encarga de traducir de lenguaje de programación de **alto nivel** a **bajo nivel**. Hay dos tipos:

Compilador: traduce el código fuente (codificado por el programador) a código máquina. **Coge el conjunto entero del código** y lo traduce en una única vez al código máquina. Java es compilado, por ejemplo.

Intérprete: traduce **línea a línea**, detiene la traducción y se ejecuta. Traducir línea por línea los puede hacer más lentos que los compilados. Python es interpretado, por ejemplo.

Como consecuencia, el programa compilado se ejecuta sin el traductor. En contraste, el interpretado necesita el intérprete **cada vez** que arranca.

CÓDIGO FUENTE, CÓDIGO OBJETO Y CÓDIGO EJECUTABLE

Código fuente: escrito en el **lenguaje de programación** que utilicemos; lo escribe el programador.

Código objeto: el **resultado de finalizar el proceso de compilación**; ya está traducido a lenguaje máquina, pero todavía no es un programa completo.

Código ejecutable: **enlaza el código objeto con las librerías** necesarias mediante el **enlazador o linker**); es el único que puede entrar en el procesador para ser ejecutado.

FASES DE CONSTRUCCIÓN DE UN PROGRAMA

El compilador tampoco traduce todo de golpe. Por dentro tiene **etapas** (analizador lexicográfico o scanner, analizador sintáctico y semántico o parser, generador de código intermedio, optimizador de código intermedio y generador de código de objeto). Pero todo esto es invisible para nosotros.